

TP 1 : CORRECTION

```
#include <stdio.h>
#include <stdlib.h>

// Définition de la structure Noeud
typedef struct Noeud
{
    int valeur;           // Valeur du noeud
    struct Noeud *suivant; // Pointeur vers le noeud suivant
} Noeud;

// Fonction pour créer un nouveau noeud
Noeud *cree(int _value)
{
    Noeud *NV;           // Pointeur vers le nouveau noeud
    NV = (Noeud *)malloc(sizeof(Noeud)); // Allocation de mémoire pour le nouveau
noeud
    NV->valeur = _value; // Initialisation de la valeur du noeud
    NV->suivant = NULL;  // Initialisation du pointeur suivant à
NULL
    return NV;           // Retourne le nouveau noeud
}

// Fonction pour afficher les valeurs de la liste
void affiche(Noeud *debut)
{
    Noeud *courant; // Pointeur pour parcourir la liste
    courant = debut; // Commence au début de la liste
    while (courant != NULL)
    {
        // Parcourt la liste jusqu'à
la fin
        printf("la valeur est %d\n", courant->valeur); // Affiche la valeur du noeud
courant
        courant = courant->suivant; // Passe au noeud suivant
    }
}

// Fonction pour ajouter un noeud au début de la liste
Noeud *ajouterD(Noeud *debut, int _value)
{
    Noeud *NV;           // Pointeur vers le nouveau noeud
    NV = (Noeud *)malloc(sizeof(Noeud)); // Allocation de mémoire pour le nouveau
noeud
    NV->valeur = _value; // Initialisation de la valeur du noeud
    NV->suivant = NULL;  // Initialisation du pointeur suivant à
NULL
    if (debut == NULL)
    {
        // Si la liste est vide
        debut = NV; // Le nouveau noeud devient le début de la liste
    }
    NV->suivant = debut; // Le nouveau noeud pointe vers l'ancien début
```

```

    debut = NV;           // Le nouveau noeud devient le nouveau début
    return debut;        // Retourne le nouveau début de la liste
}

// Fonction pour ajouter un noeud à la fin de la liste
Noeud *ajouterF(Noeud *debut, int _value)
{
    Noeud *NV;            // Pointeur vers le nouveau noeud
    NV = (Noeud *)malloc(sizeof(Noeud)); // Allocation de mémoire pour le nouveau
noeud
    NV->valeur = _value;   // Initialisation de la valeur du noeud
    NV->suivant = NULL;    // Initialisation du pointeur suivant à
NULL
    if (debut == NULL)
    {
        // Si la liste est vide
        debut = NV; // Le nouveau noeud devient le début de la liste
    }
    Noeud *courant; // Pointeur pour parcourir la liste
    courant = debut; // Commence au début de la liste
    while (courant->suivant != NULL)
    {
        // Parcourt jusqu'à la fin de la liste
        courant = courant->suivant; // Passe au noeud suivant
    }
    courant->suivant = NV; // Ajoute le nouveau noeud à la fin
    return debut;        // Retourne le début de la liste
}

// Fonction pour rechercher une valeur dans la liste
void rechercher(Noeud *debut, int _value)
{
    int trouve;           // Variable pour indiquer si la valeur est trouvée
    Noeud *courant; // Pointeur pour parcourir la liste
    courant = debut; // Commence au début de la liste
    while (courant != NULL)
    { // Parcourt la liste jusqu'à la fin
        if (courant->valeur == _value)
        {
            // Si la valeur est trouvée
            trouve = 1; // Marque comme trouvée
            break;      // Sort de la boucle
        }
        else
        {
            trouve = 0; // Marque comme non trouvée
        }
        courant = courant->suivant; // Passe au noeud suivant
    }
    if (trouve == 1) // Si la valeur est trouvée
        printf("la valeur existe\n");
    else // Si la valeur n'est pas trouvée
        printf("la valeur n'existe pas\n");
}

```

```
// Fonction pour calculer la taille de la liste
int taille(Noeud *debut)
{
    int taille = 0; // Variable pour stocker la taille de la liste
    Noeud *courant; // Pointeur pour parcourir la liste
    courant = debut; // Commence au début de la liste
    while (courant != NULL)
    {
        // Parcourt la liste jusqu'à la fin
        taille++; // Incrémente la taille
        courant = courant->suivant; // Passe au noeud suivant
    }
    return taille; // Retourne la taille de la liste
}

// Fonction pour insérer un noeud à une position donnée
Noeud *insertion(Noeud *debut, int pos, int val)
{
    Noeud *NV, *courant; // Pointeurs pour le nouveau noeud et pour
    parcourir la liste
    NV = (Noeud *)malloc(sizeof(Noeud)); // Allocation de mémoire pour le nouveau
    noeud
    NV->valeur = val; // Initialisation de la valeur du noeud
    NV->suivant = NULL; // Initialisation du pointeur suivant à
    NULL
    int size = taille(debut); // Calcule la taille de la liste
    if (pos < 1 || pos > size + 1)
    { // Vérifie si la position est valide
        printf("la position est incorrecte\n");
    }
    else if (pos == 1)
    {
        // Si la position est le début de la liste
        NV->suivant = debut; // Le nouveau noeud pointe vers l'ancien début
        debut = NV; // Le nouveau noeud devient le nouveau début
    }
    else
    {
        courant = debut; // Commence au début de la liste
        for (int i = 1; i < pos - 1; i++)
        {
            // Parcourt jusqu'à la position précédente
            courant = courant->suivant; // Passe au noeud suivant
        }
        NV->suivant = courant->suivant; // Le nouveau noeud pointe vers le noeud
        suivant
        courant->suivant = NV; // Le noeud précédent pointe vers le nouveau
        noeud
    }
    return debut; // Retourne le début de la liste
}

// Fonction pour supprimer le premier noeud de la liste
```

```
Noeud *suppressionD(Noeud *debut)
{
    Noeud *P; // Pointeur vers le noeud à supprimer
    P = debut; // Pointe vers le début de la liste
    if (debut == NULL)
    { // Si la liste est vide
        printf("la liste est vide\n");
    }
    else
    {
        debut = debut->suivant; // Le début de la liste devient le deuxième noeud
        free(P); // Libère la mémoire du premier noeud
    }
    return debut; // Retourne le nouveau début de la liste
}

// Fonction pour supprimer le dernier noeud de la liste
Noeud *suppressionF(Noeud *debut)
{
    if (debut == NULL) // Si la liste est vide
        printf("la liste est vide\n");
    Noeud *courant; // Pointeur pour parcourir la liste
    courant = debut; // Commence au début de la liste
    if (courant->suivant == NULL)
    { // Si la liste contient un seul noeud
        free(courant); // Libère la mémoire du noeud
        printf("l'element a été bien supprimé\n");
    }
    while (courant->suivant->suivant != NULL)
    { // Parcourt jusqu'à l'avant-dernier noeud
        courant = courant->suivant; // Passe au noeud suivant
    }
    free(courant->suivant); // Libère la mémoire du dernier noeud
    courant->suivant = NULL; // L'avant-dernier noeud pointe maintenant vers NULL
    return debut; // Retourne le début de la liste
}

// Fonction pour supprimer un noeud à une position donnée
Noeud *suppressionP(Noeud *debut, int pos)
{
    Noeud *courant, *tmp; // Pointeurs pour parcourir la liste et pour le noeud à supprimer
    courant = debut; // Commence au début de la liste
    tmp = debut; // Initialisation du pointeur temporaire
    int i; // Variable de boucle
    if (debut == NULL)
    { // Si la liste est vide
        printf("la liste est vide\n");
        return NULL;
    }
    int size = taille(debut); // Calcule la taille de la liste
```

```
if (pos < 1 || pos > size)
{ // Vérifie si la position est valide
    printf("la position est invalide\n");
    return debut;
}
if (pos == 1)
{
    // Si la position est le début de la liste
    debut = debut->suivant; // Le début de la liste devient le deuxième noeud
    free(courant);          // Libère la mémoire du premier noeud
    return debut;          // Retourne le nouveau début de la liste
}
for (i = 1; i < pos - 1; i++)
{
    // Parcourt jusqu'à la position précédente
    courant = courant->suivant; // Passe au noeud suivant
}
tmp = courant->suivant; // Pointe vers le noeud à supprimer
courant->suivant = tmp->suivant; // Le noeud précédent pointe vers le noeud
suivant
free(tmp); // Libère la mémoire du noeud à supprimer
return debut; // Retourne le début de la liste
}

// Fonction pour modifier la valeur du premier noeud
Noeud *modifierD(Noeud *debut, int _value)
{
    if (debut == NULL) // Si la liste est vide
        return NULL;
    debut->valeur = _value; // Modifie la valeur du premier noeud
    return debut;          // Retourne le début de la liste
}

// Fonction pour modifier la valeur du dernier noeud
Noeud *modifierF(Noeud *debut, int _value)
{
    if (debut == NULL)
    { // Si la liste est vide
        printf("la liste est vide\n");
        return NULL;
    }
    Noeud *courant; // Pointeur pour parcourir la liste
    courant = debut; // Commence au début de la liste
    while (courant->suivant != NULL)
    {
        // Parcourt jusqu'à la fin de la liste
        courant = courant->suivant; // Passe au noeud suivant
    }
    courant->valeur = _value; // Modifie la valeur du dernier noeud
    return debut;          // Retourne le début de la liste
}

// Fonction pour modifier la valeur d'un noeud à une position donnée
Noeud *modifierP(Noeud *debut, int pos, int _value)
```

```
{
    int i; // Variable de boucle
    if (debut == NULL)
    { // Si la liste est vide
        printf("la liste est vide\n");
        return NULL;
    }
    int size = taille(debut); // Calcule la taille de la liste
    if (pos < 1 || pos > size)
    { // Vérifie si la position est valide
        printf("la position est invalide\n");
        return debut;
    }
    Noeud *courant; // Pointeur pour parcourir la liste
    courant = debut; // Commence au début de la liste
    for (i = 1; i < pos; i++)
    {
        // Parcourt jusqu'à la position donnée
        courant = courant->suivant; // Passe au noeud suivant
    }
    courant->valeur = _value; // Modifie la valeur du noeud à la position donnée
    return debut; // Retourne le début de la liste
}

// Fonction principale
int main()
{
    Noeud *debut = NULL; // Pointeur vers le début de la liste
    Noeud *P = cree(10); // Crée un noeud avec la valeur 10
    Noeud *D = cree(20); // Crée un noeud avec la valeur 20
    Noeud *T = cree(30); // Crée un noeud avec la valeur 30

    printf("adresse du noeud debut : %p\n", debut);
    printf("adresse du noeud P : %p\n", P);
    printf("adresse du noeud D : %p\n", D);
    printf("adresse du noeud T : %p\n", T);

    debut = P; // Début de la liste pointe vers P
    P->suivant = D; // P pointe vers D
    D->suivant = T; // D pointe vers T
    T->suivant = NULL; // T pointe vers NULL (fin de la liste)

    printf("-----\n");
    printf("adresse du noeud %p\n", debut);
    printf("adresse du noeud P suivant %p\n", P->suivant);
    printf("adresse du noeud D suivant %p\n", D->suivant);
    printf("adresse du noeud T suivant %p\n", T->suivant);

    printf("-----Affichage-----\n");
    affiche(debut); // Affiche la liste

    printf("-----Ajout au debut-----\n");
```

```
debut = ajouterD(debut, 100); // Ajoute un noeud au début de la liste
affiche(debut);              // Affiche la liste

printf("-----Affichage ajout à la Fin-----\n");
debut = ajouterF(debut, 200); // Ajoute un noeud à la fin de la liste
affiche(debut);              // Affiche la liste

printf("-----Recherche-----\n");
rechercher(debut, 20); // Recherche une valeur dans la liste

printf("-----Taille de la liste-----\n");
printf("la taille est %d\n", taille(debut)); // Affiche la taille de la liste

printf("-----Insertion à une position-----\n");
debut = insertion(debut, 3, 555); // Insère un noeud à une position donnée
affiche(debut);                  // Affiche la liste

printf("-----Suppression au debut-----\n");
debut = suppressionD(debut); // Supprime le premier noeud de la liste
affiche(debut);              // Affiche la liste

printf("-----Suppression à la fin-----\n");
debut = suppressionF(debut); // Supprime le dernier noeud de la liste
affiche(debut);              // Affiche la liste

printf("-----Suppression à une position-----\n");
debut = suppressionP(debut, 4); // Supprime un noeud à une position donnée
affiche(debut);                  // Affiche la liste

printf("-----Modifier au debut-----\n");
debut = modifierD(debut, 2024); // Modifie la valeur du premier noeud
affiche(debut);                  // Affiche la liste

printf("-----Modifier à la fin-----\n");
debut = modifierF(debut, 2025); // Modifie la valeur du dernier noeud
affiche(debut);                  // Affiche la liste

printf("-----Modifier à une position-----\n");
debut = modifierP(debut, 1, 2000); // Modifie la valeur d'un noeud à une
position donnée
affiche(debut);                  // Affiche la liste

return 0; // Fin du programme
}
```